

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 03 • NIVEL 3 - AVANZADO

Clase 03: Salidas Estructuradas y Validación Tipada con Pydantic V2

«Pydantic como la Aduana Estricta de Datos para Respuestas de IA»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python
mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 03** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

- Competencia Conceptual:** Comprender Structured Outputs, esquemas JSON Schema generados por Pydantic y serialización estricta.
- Competencia Práctica:** Garantizar que las respuestas del LLM cumplan con un modelo de datos tipado antes de entrar a la base de datos.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 03: Salidas Estructuradas y Validación Tipada con Pydantic V2

Integrar LLMs en sistemas empresariales exige que sus respuestas sean 100% deterministas en estructura y tipo.

☀ Metáfora Central: Pydantic como la Aduana Estricta de Datos para Respuestas de IA

Pydantic es el inspector de aduana que revisa que cada paquete traiga exactamente los sellos, tipos y formatos requeridos.

Principios Teóricos y Modelo Mental

Pydantic V2 está construido sobre un núcleo en Rust de alto rendimiento para validación instantánea.

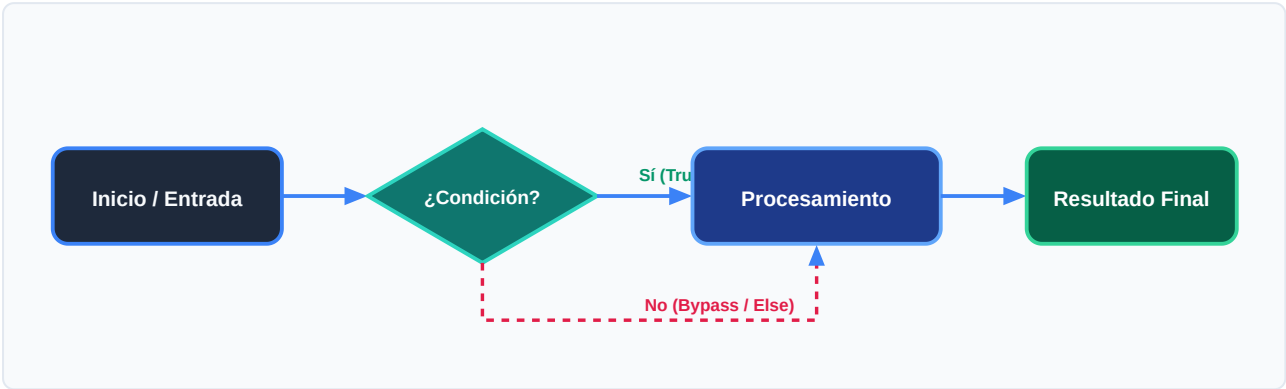
Structured Outputs obliga al LLM a seguir la gramática JSON Schema del modelo BaseModel.

⚡ Regla de Oro en Python

Nunca consumas texto libre de un LLM en lógica transaccional; valida siempre con Pydantic.

Arquitectura y Movimiento de Datos en Memoria

Flujo de inferencia con esquema JSON y validación Pydantic.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Definición del modelo BaseModel con campos y validadores Field().	Esquema JSON Schema exportado.
2. Evaluación	Inyección del esquema en la llamada de API del LLM.	Inferencia restringida por gramática.
3. Transformación	Recepción del payload JSON crudo.	String JSON recibido.
4. Retorno / Salida	Validación con Modelo.model_validate_json().	Objeto Python fuertemente tipado instanciado.

Visualización Mental

Si el JSON del LLM tiene un campo faltante o tipo incorrecto, Pydantic lanza `ValidationError` inmediatamente.

Código de Demostración en Python 3.10+

Extracción tipada de entidades con Pydantic V2:

main.py (Python 3.10+)

PEP 8 Compliant

```
from pydantic import BaseModel, Field, EmailStr

class LeadCliente(BaseModel):
    nombre: str = Field(description="Nombre completo del prospecto")
    email: str = Field(description="Correo electrónico válido")
    presupuesto_estimado: float = Field(ge=0.0, description="Monto en USD")
    interes_ia: bool = True

# Simulación de respuesta JSON generada por LLM
json_llm = '{"nombre": "Laura Méndez", "email": "laura@empresa.com", "presupuesto_estimado": 15000.0}'
lead = LeadCliente.model_validate_json(json_llm)

print("Lead Validado:", lead.nombre)
print("Presupuesto:", lead.presupuesto_estimado)
```

Análisis de Ingeniería del Código

Uso de Field con descripciones semánticas y restricciones numéricas ge=0.0.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Errores de serialización en respuestas no conformes.

⚠ Gotcha Frecuente (Trampa de Principiante)

Parsear la salida del LLM con `json.loads()` simple sin validar tipos permite que campos nulos rompan la aplicación.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
data = json.loads(respuesta_llm)
total = data['precio'] * 2 # ❌ Falla si
'precio' vino como None o string
```

✅ Patrón Correcto

```
data =
FacturaModel.model_validate_json(respuesta_llm)
total = data.precio * 2 # ✅ Garantizado float
tipado
```

🛡 Consejo de Resiliencia en Producción

Pydantic permite definir validadores personalizados con el decorador `@field_validator`.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 03: Salidas Estructuradas y Validación Tipada con Pydantic V2**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Pydantic como la Aduana Estricta de Datos para Respuestas de IA
2. Regla de Oro	Nunca consumas texto libre de un LLM en lógica transaccional; valida siempre con Pydantic.
3. Gotcha Crítico	Parsear la salida del LLM con <code>json.loads()</code> simple sin validar tipos permite que campos nulos rompan la aplicación.
4. Buenas Prácticas	Pydantic permite definir validadores personalizados con el decorador <code>@field_validator</code> .

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea un modelo Pydantic para validar órdenes de compra con lista de productos, impuestos y total.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.